



MISSION MONGODB PEPINOS



Estructura del Proyecto

```
MISSION-MONGODB-PEPINOS/
├── src/
│   ├── mision-01-mongodb/
│   │   ├── ANALISIS_NOSQL.md
│   │   ├── ANALISIS_NOSQL.pdf
│   │   ├── misiones-mongodb.js
│   │   ├── README.md
│   │   └── README.pdf
│   ├── mision-02-validacion/
│   │   ├── 01-guardianes.mongodb
│   │   ├── 02_definicion_criaturas.mongodb
│   │   ├── 03_pruebas_insercion.mongodb
│   │   ├── ANALISIS_VALIDACION.md
│   │   ├── README.md
│   │   └── README.pdf
├── .editorconfig
├── .gitignore
├── docker-compose.yml
└── README.md ← Archivo principal con la descripción general del proyecto
```



TAREA 2 EL CRONISTA DE DATOS NOSQL

Base de Datos **NoSQL** Herramienta **VSCode** Entorno **MongoDB Atlas** Autor **Pepinós Arboleda Brian**



Autor

Nombre completo: TNTE Pepinós Arboleda Brian

Asignatura: Modelado avanzado de Base de Datos

Fecha: 20 Octubre 2025

Repositorio de github:

<https://github.com/BRIANPEPINOS/mision-mongodb-pepinos>



El Bestiario Digital

En esta misión, el cronista de datos ha sido convocado para construir un Bestiario Digital de Criaturas Fantásticas, una colección donde cada ser mítico posee atributos únicos: alas, fuego, hechizos o escamas.

A diferencia de una base de datos relacional, donde cada campo debe ser rígido y uniforme, MongoDB permite representar estos seres mediante un esquema flexible y natural. Cada criatura es un documento independiente, con estructuras personalizadas y atributos anidados, demostrando el verdadero poder del modelo NoSQL.

⚙️ Instrucciones para ejecutar el script

El archivo principal de esta misión es `misiones_mongodb.js`, el cual contiene todas las operaciones CRUD necesarias para crear, leer, actualizar y analizar el Bestiario Digital.

◇ Opción 1. Ejecución en MongoDB Atlas (en la nube)

1. Inicia sesión en [MongoDB Atlas](#) y abre tu cluster.
 2. Haz clic en **Connect** → **Connect with Mongo Shell** y copia tu cadena de conexión.
Ejemplo: `mongosh "mongodb+srv://.mongodb.net/?retryWrites=true&w=majority" --username`
 3. Una vez dentro de `mongosh`, ejecuta: `load('misiones-mongodb.js')`
 4. Se creará la base de datos **bestiario** con la colección **criaturas** y se mostrarán los resultados en pantalla.
-

◇ Opción 2. Ejecución local (MongoDB Community Server)

1. Asegúrate de tener **MongoDB** y **mongosh** instalados en tu sistema.
 2. Copia el archivo `misiones-mongodb.js` dentro de tu carpeta de trabajo.
 3. Abre una terminal y ejecuta: `mongosh load('misiones_mongodb.js')`
 4. Verás los mensajes del script confirmando las inserciones y actualizaciones realizadas.
-

◇ Opción 3. Ejecución desde VS Code (Extensión MongoDB Playground) 🧪

Esta es la forma más práctica si trabajas directamente desde Visual Studio Code.

🔧 Pasos:

1. Instala la extensión oficial **MongoDB for VS Code** (autor: MongoDB Inc).
2. Abre la barra lateral de MongoDB (icono verde 🟢 a la izquierda).
3. Conéctate a tu **cluster de Atlas** o instancia local.
4. Haz clic en **"Create New Playground"**.
Esto abrirá un archivo temporal con el nombre `Playground-0.mongodb.js`.
5. Elimina el contenido por defecto y **pega todo el código** de tu archivo `misiones_mongodb.js`.
6. En la esquina superior derecha, verifica que diga:
☒ **Connected to tu cluster**
7. Finalmente, haz clic en el botón ▶ **Run** (o presiona `Ctrl + Shift + R`).

📄 Operaciones realizadas en el script

Tipo	Descripción	Ejemplo
 Create	Inserta criaturas con estructuras variadas usando <code>insertOne()</code> y <code>insertMany()</code>	<code>db.criaturas.insertOne({...})</code>
 Read	Consulta todas las criaturas, por hábitat o por nivel de peligro	<code>db.criaturas.find({ habitat: "Bosque Encantado" })</code>
 Update	Añade habilidades o incrementa niveles de peligro con <code>updateOne()</code> y <code>updateMany()</code>	<code>db.criaturas.updateMany({ habitat: "Bosque Encantado" }, { \$inc: { nivel_peligro: 1 } })</code>



TAREA 3 EL GUARDIÁN DEL ESQUEMA Y LOS VÍNCULOS ARCANOS (Misión 2)

Base de Datos

NoSQL

Herramienta

VSCode

Entorno

MongoDB Local

Misión

2



Descripción de la Misión

En esta segunda misión, el Guardián del Esquema fue llamado para proteger la pureza de los datos del *Bestiario Digital*.

El propósito fue implementar reglas de validación usando **JSON Schema** y modelar las relaciones entre colecciones, tanto embebidas como referenciadas.

Con esto, se fortaleció la base de datos asegurando que ningún documento inválido pudiera ser insertado, manteniendo la consistencia entre guardianes y criaturas.



Estructura de Archivos

Archivo	Descripción
<code>01_definicion_guardianes.mongodb</code>	Define la colección <code>guardianes</code> con validaciones estrictas y un inventario embebido.
<code>02_definicion_criaturas.mongodb</code>	Define la colección <code>criaturas</code> con reglas de validación, ficha veterinaria embebida y relación referenciada con guardianes.
<code>03_pruebas_insercion.mongodb</code>	Contiene pruebas de integridad, incluyendo inserciones válidas e inválidas con sus respectivos errores.
<code>ANALISIS_VALIDACION.md</code>	Documento teórico con el análisis del modelo, validación y justificación de las relaciones.

Todos los archivos se ejecutan desde la carpeta `misión_02_validacion/` usando la extensión **MongoDB for VS Code**.

🌟 Instrucciones para ejecutar los archivos en VS Code

1. Abre **Visual Studio Code** y asegúrate de tener instalada la extensión oficial **MongoDB for VS Code**.
 2. Conéctate a tu base de datos local o a tu cluster de Atlas.
 3. Abre cada archivo `.mongodb` en el orden indicado:
 - `01_definicion_guardianes.mongodb`
 - `02_definicion_criaturas.mongodb`
 - `03_pruebas_insercion.mongodb`
 4. Verifica que aparezca en la esquina superior derecha:
☒ **Connected to Cluster / localhost**
 5. Ejecuta con el botón ► **Run Playground** o presionando `Ctrl + Shift + R`.
 6. Observa los resultados de las validaciones y errores directamente en la consola del Playground.
-

🌸 Pruebas de Integridad

Durante las pruebas, se realizaron inserciones válidas e inválidas para comprobar las reglas de validación implementadas en las colecciones `guardianes` y `criaturas`.

MongoDB respondió con el código de error **121 (Document failed validation)** en los casos donde los documentos no cumplían las condiciones del esquema, confirmando que las restricciones funcionaban correctamente.

Los mensajes de error fueron registrados dentro del archivo `03_pruebas_insercion.mongodb`, dejando evidencia del blindaje de datos aplicado por el Guardián del Esquema.

🧠 Resultados del Modelado

- Las relaciones **embebidas** (Guardián → Inventario y Criatura → Ficha Veterinaria) mejoraron la coherencia y las operaciones atómicas.
 - Las relaciones **referenciadas** (Guardián → Criaturas) optimizaron el rendimiento y la escalabilidad del sistema.
 - El uso de **JSON Schema** permitió validar tipos, rangos, enumeraciones y patrones de texto directamente desde la base de datos.
 - Las estructuras con subdocumentos y arreglos mostraron un comportamiento estable, facilitando las operaciones de lectura y actualización sin afectar la integridad de los datos.
-